

Documento funzionale - Gestionale programmazione cinema multisala

1. Contesto del progetto

Un cinema multisala vuole realizzare un gestionale interno per organizzare la propria programmazione settimanale.

Il sistema dovrà permettere allo staff di gestire:

- il catalogo dei film disponibili;
- le sale del cinema;
- le proiezioni programmate;
- eventi speciali e proiezioni 3D;
- ricerche e report sulla programmazione;
- informazioni basate su date e orari, come proiezioni future, proiezioni di oggi, orari di fine e proiezioni serali.

2. Obiettivo

La prima versione deve permettere di caricare alcuni film, alcune sale e diverse proiezioni, per poi ottenere report utili alla gestione quotidiana del cinema.

Il personale del cinema vuole poter sapere, ad esempio:

- quali film sono presenti nel catalogo;
- quali proiezioni sono previste in una certa data;
- quali proiezioni sono in programma in una determinata sala;
- quali proiezioni sono serali;
- quali proiezioni sono future;
- quali film appartengono a un certo genere;
- quali proiezioni sono eventi speciali o proiezioni 3D;
- quali proiezioni hanno prezzo più alto;
- a che ora termina una proiezione in base alla durata del film.

3. Vincoli tecnici richiesti

Il progetto deve essere sviluppato in **Java**.

In caso di dati non validi, il programma deve semplicemente stampare un messaggio e non completare l'operazione, oppure mantenere il valore precedente.

4. Gestione del catalogo film

Il cinema deve poter registrare diversi film nel proprio catalogo.

Ogni film deve avere almeno queste informazioni:

- identificativo numerico univoco;
- titolo;
- regista;
- durata in minuti;
- data di uscita;
- uno o più generi.

I generi devono essere gestiti con un **enum**, non come semplici stringhe libere.

Esempi di generi possibili:

AZIONE
COMMEDIA
DRAMMATICO
THRILLER
FANTASCIENZA
HORROR
ANIMAZIONE
DOCUMENTARIO

Un film può appartenere a più generi.

Lo stesso genere non deve essere duplicato per lo stesso film, quindi è opportuno usare una struttura dati che impedisca duplicati.

Il sistema deve permettere di:

- aggiungere un genere a un film;
- verificare se un film appartiene a un certo genere;
- stampare una descrizione leggibile del film;
- sapere da quanti anni il film è uscito;
- sapere se un film è recente.

Per questa versione, un film può essere considerato recente se è uscito negli ultimi 2 anni rispetto alla data attuale.

5. Gestione delle sale

Il cinema è composto da più sale.

Ogni sala deve avere almeno:

- identificativo numerico univoco;
- nome della sala;
- numero massimo di posti;
- indicazione se supporta il 3D;
- elenco di caratteristiche tecniche o commerciali.

Esempi di caratteristiche:

IMAX

Dolby Atmos

Poltrone reclinabili

Accessibile disabili

Schermo grande

Anche le caratteristiche non devono essere duplicate per la stessa sala.

Il sistema deve permettere di:

- aggiungere una caratteristica alla sala;
- verificare se una sala possiede una certa caratteristica;
- stampare una descrizione sintetica della sala.

6. Gestione delle proiezioni

Il cuore del gestionale è la programmazione delle proiezioni.

Una proiezione rappresenta un film programmato in una certa sala, in una certa data e a un certo orario.

Ogni proiezione deve avere almeno:

- identificativo numerico univoco;
- film associato;
- sala associata;
- data della proiezione;
- ora di inizio;
- prezzo base;
- eventuali tag descrittivi.

Esempi di tag:

SERALE
WEEKEND
FAMIGLIE
ANTEPRIMA
LINGUA_ORIGINALE
EVENTO
TRE_D

I tag possono anche essere gestiti come **String**, perché in questo caso possono essere più liberi rispetto ai generi dei film.

Anche qui, però, non devono esserci duplicati.

7. Tipologie di proiezione

Il cinema non organizza solo proiezioni normali.

Il sistema deve distinguere almeno tre tipologie di proiezione.

Proiezione standard

È una normale proiezione di un film.

Il prezzo finale parte dal prezzo base e può aumentare leggermente in base ad alcune condizioni:

- se la proiezione è nel weekend;
- se la proiezione è serale.

Proiezione 3D

È una proiezione che prevede contenuti in 3D.

Oltre alle informazioni comuni di una proiezione, deve gestire:

- supplemento 3D;
- indicazione se gli occhiali sono inclusi.

Il prezzo finale deve tenere conto:

- del prezzo base;
- del supplemento 3D;
- dell'eventuale costo aggiuntivo se gli occhiali non sono inclusi;

- dell'eventuale maggiorazione weekend.

Il sistema deve anche controllare che la sala scelta supporti il 3D.

Se la sala non supporta il 3D, il programma deve stampare un messaggio di avviso.

Evento speciale

Il cinema organizza anche eventi speciali, come:

- anteprime;
- rassegne;
- film restaurati;
- proiezioni con ospite;
- incontri con il regista.

Un evento speciale deve avere almeno:

- nome dell'evento;
- eventuale ospite;
- indicazione se i posti sono limitati.

Il prezzo finale deve tenere conto:

- del prezzo base;
- di una maggiorazione per evento speciale;
- di un'eventuale maggiorazione per posti limitati;
- di un'eventuale maggiorazione serale.

8. Gestione di date e orari

Il sistema deve usare correttamente le classi Java per date e orari.

Per i film è sufficiente usare:

`LocalDate`

Per le proiezioni bisogna usare:

`LocalDate`

`LocalTime`

`LocalDateTime`

Il sistema deve essere in grado di calcolare:

- data e ora di inizio della proiezione;

- data e ora di fine della proiezione;
- se una proiezione è programmata per oggi;
- se una proiezione è nel weekend;
- se una proiezione è serale;
- se una proiezione è già terminata;
- se una proiezione è futura.

Una proiezione è serale se inizia dalle **20:00** in poi.

Una proiezione è terminata se la sua data e ora di fine sono precedenti alla data e ora attuale.

L'orario di fine deve essere calcolato sommando la durata del film all'orario di inizio della proiezione.

9. Archiviazione degli oggetti

Il cliente vuole poter recuperare velocemente film, sale e proiezioni tramite il loro identificativo.

Per questo motivo, il progetto deve prevedere un piccolo archivio generico riutilizzabile per oggetti identificabili. Non deve esistere un archivio diverso scritto da zero per ogni tipo di oggetto.

L'archivio deve poter contenere:

- film;
- sale;
- proiezioni.

Ogni elemento salvato nell'archivio deve avere un id.

L'archivio deve permettere di:

- aggiungere un elemento;
- cercare un elemento per id;
- rimuovere un elemento per id;
- restituire tutti gli elementi;
- contare quanti elementi contiene.

Internamente, l'archivio dovrebbe usare una **Map**, perché il recupero per id è l'operazione più importante.

Lettura dati da file

Il progetto deve prevedere una classe dedicata alla lettura dei dati da file.

La classe può chiamarsi:

LettoreDatiCinema

Questa classe deve occuparsi di:

- leggere i film da file;
- leggere le sale da file;
- leggere le proiezioni da file;
- creare gli oggetti corrispondenti;
- aggiungerli al **GestoreCinema**;
- ignorare le righe non valide, stampando un messaggio.

File **film.txt**

Formato richiesto:

id;titolo;regista;durataMinuti;dataUscita;generi

Esempio:

1;Dune Part Two;Denis Villeneuve;166;2024-03-01;FANTASCIENZA,AZIONE

2;Oppenheimer;Christopher Nolan;180;2023-08-23;DRAMMATICO,STORICO

3;Inside Out 2;Kelsey Mann;96;2024-06-19;ANIMAZIONE,COMMEDIA

I generi devono corrispondere ai valori dell'enum **GenereFilm**.

File **sale.txt**

Formato richiesto:

id;nome;numeroPosti;supporta3D;caratteristiche

Esempio:

1;Sala IMAX;220;true;IMAX,DOLBY_ATMOS,SCHERMO_GRANDE

2;Sala Blu;120;false;ACCESSIBILE_DISABILI,POLTRONE_RECLINABILI

3;Sala 3D;160;true;DOLBY_ATMOS,SCHERMO_GRANDE

Il campo **supporta3D** può essere: true o false

Le caratteristiche possono restare **String**, quindi non serve per forza un enum.

File **proiezioni.txt**

Per le proiezioni conviene usare un solo file con un campo **tipo**.

Formato richiesto:

id;tipo;filmId;salaId;data;oraInizio;prezzoBase;extra1;extra2;extra3;tag

Il campo **tipo** può essere:

STANDARD

TRE_D

EVENTO

Esempi:

1;STANDARD;1;1;2026-06-10;21:30;10.00;-;-;SERALE,WEEKEND

2;TRE_D;1;3;2026-06-11;20:15;12.00;3.00;true;-;TRE_D,SERALE

3;EVENTO;2;1;2026-06-12;19:00;14.00;Anteprima speciale;Regista ospite;true;EVENTO,ANTEPRIMA

Significato dei campi extra:

Per **STANDARD**:

extra1 = -

extra2 = -

extra3 = -

Per **TRE_D**:

extra1 = supplemento3D

extra2 = occhialiInclusi

extra3 = -

Per **EVENTO**:

extra1 = nomeEvento

extra2 = ospite

extra3 = postiLimitati

Metodi da aggiungere

Nella classe `LettoreDatiCinema` puoi prevedere metodi di questo tipo:

```
public List<Film> leggiFilm(String percorsoFile)
```

```
public List<Sala> leggiSale(String percorsoFile)
```

```
public List<Proiezione> leggiProiezioni( String percorsoFile, Archivio<Film> archivioFilm,  
Archivio<Sala> archivioSale )
```

Controlli richiesti durante la lettura

Durante la lettura dei file devi controllare che:

- la riga non sia vuota;
- la riga abbia il numero corretto di campi;
- l'id sia numerico;
- la durata sia maggiore di 0;
- il prezzo base sia maggiore di 0;
- la data sia nel formato `yyyy-MM-dd`;
- l'ora sia nel formato `HH:mm`;
- il genere esista nell'enum `GenereFilm`;
- il tipo di proiezione sia valido;
- il film indicato da `filmId` esista;
- la sala indicata da `salaId` esista.

Se una riga non è valida, il programma non deve bloccarsi.

Deve stampare un messaggio tipo:

Riga non valida nel file proiezioni.txt: film con id 8 non trovato.

e continuare con la riga successiva.

10. Gestore principale del cinema

Il sistema deve prevedere una classe principale di gestione, che rappresenti il servizio interno del cinema.

Questa classe deve coordinare:

- catalogo film;
- sale;
- programmazione;
- ricerche;
- report;
- filtri;
- ordinamenti.

Il gestore deve mantenere:

- un archivio dei film;
- un archivio delle sale;
- un archivio delle proiezioni;
- una lista completa della programmazione;
- una struttura per raggruppare le proiezioni per data;
- una struttura per raggruppare le proiezioni per sala;
- una struttura per raggruppare i film per genere.

Quindi, anche se non vengono indicati direttamente i nomi delle classi, il progetto deve distinguere chiaramente tra:

- oggetti del dominio;
- archivio generico;
- classe di servizio/gestione;
- classe **Main** per il test.

11. Funzionalità richieste dal cliente

Inserimento dati

Il sistema deve permettere di registrare:

- nuovi film;
- nuove sale;
- nuove proiezioni.

Quando viene aggiunto un film, il sistema deve aggiornare anche il raggruppamento dei film per genere.

Quando viene aggiunta una proiezione, il sistema deve aggiornare:

- la lista completa della programmazione;
- il raggruppamento per data;
- il raggruppamento per sala.

Ricerca delle proiezioni

Lo staff deve poter cercare:

- tutte le proiezioni di una certa data;
- tutte le proiezioni di una certa sala;
- tutte le proiezioni previste per oggi;
- tutte le proiezioni future;
- tutte le proiezioni serali;
- tutte le proiezioni del weekend;
- tutte le proiezioni con prezzo superiore a una certa soglia.

Ricerca dei film

Lo staff deve poter cercare:

- tutti i film di un certo genere;
- tutti i film recenti;
- tutti i film con durata superiore a un certo numero di minuti.

12. Filtri personalizzati

Il cliente vorrebbe che il sistema fosse abbastanza flessibile da poter applicare filtri diversi senza creare ogni volta un metodo nuovo.

Per questo motivo, il progetto deve prevedere un piccolo meccanismo di filtro generico.

Il filtro deve poter essere applicato almeno alle proiezioni e, se vuoi, anche ai film.

Esempi di filtri richiesti:

- proiezioni con prezzo finale maggiore di 15;
- proiezioni nel weekend;

- proiezioni che sono eventi speciali;
- film recenti;
- film più lunghi di 140 minuti;
- film di un certo genere.

Questi filtri devono essere usati nel `main` tramite `lambda`.

13. Ordinamenti richiesti

Il sistema deve permettere di visualizzare i dati anche in ordine diverso.

Sono richiesti almeno questi ordinamenti:

- proiezioni ordinate per data e ora di inizio;
- proiezioni ordinate per prezzo finale crescente;
- film ordinati per durata decrescente.

Gli ordinamenti devono essere realizzati con `lambda`.

14. Notifiche simulate

Il cliente vorrebbe in futuro inviare notifiche quando una nuova proiezione viene programmata.

Per ora non bisogna inviare vere email o notifiche reali.

Deve però essere simulato un sistema di notifica. La notifica deve ricevere una proiezione e stampare un messaggio.

Per esercitarti, devi creare due versioni:

- una notifica usando una classe anonima;
- una notifica usando una `lambda`.

Esempi di messaggi:

Email: nuova proiezione programmata per Dune: Part Two il 10/06/2026 alle 21:30

App: evento speciale disponibile nella Sala IMAX

15. Uso del polimorfismo

Il gestionale deve trattare tutte le proiezioni in modo uniforme quando possibile.

Per esempio, la lista della programmazione deve poter contenere insieme:

- proiezioni standard;
- proiezioni 3D;
- eventi speciali.

Questo significa che il codice dovrà ragionare spesso sul tipo generale **Proiezione**.

Tuttavia, in alcuni report il cliente vuole vedere dettagli specifici in base al tipo reale della proiezione.

Per esempio:

- per una proiezione 3D vuole vedere il supplemento e se gli occhiali sono inclusi;
- per un evento speciale vuole vedere il nome dell'evento e l'ospite;
- per una proiezione standard bastano le informazioni principali.

16. Interfacce consigliate

Il progetto dovrebbe prevedere alcune interfacce per separare meglio i comportamenti.

Per esempio, può essere utile avere un'interfaccia per gli oggetti che hanno un id, una per gli oggetti programmabili nel tempo e una per gli oggetti che calcolano un prezzo.

Non è necessario che il cliente conosca questi dettagli tecnici, ma il codice deve essere progettato in modo ordinato.

Una possibile organizzazione è:

Identificabile
Programmabile
Prezzabile
Filtro<T>
Notificatore

Queste interfacce aiutano a rendere il progetto più pulito.

17. Dati minimi da caricare nel main

Per dimostrare che il gestionale funziona, devi creare nel **main** un set di dati realistico.

Sono richiesti almeno:

- 6 film;
- 4 sale;
- 10 proiezioni.

Tra le 10 proiezioni devono esserci:

- almeno 4 proiezioni standard;
- almeno 3 proiezioni 3D;
- almeno 3 eventi speciali.

Ogni film deve avere almeno 2 generi.

Ogni sala deve avere almeno 2 caratteristiche.

Ogni proiezione deve avere almeno 1 tag.

Importazione dati da file

Per la prima versione del gestionale, il cliente non richiede ancora un database.

Tuttavia, lo staff del cinema vuole evitare di inserire tutti i dati direttamente nel codice.

Il sistema dovrà quindi permettere il caricamento iniziale dei dati tramite file testuali.

I file previsti sono:

film.txt

sale.txt

proiezioni.txt

Il sistema dovrà leggere i file all'avvio dell'applicazione e popolare automaticamente:

- catalogo film;
- elenco sale;
- programmazione delle proiezioni.

Il caricamento dovrà avvenire in ordine logico:

1. caricamento dei film;
2. caricamento delle sale;
3. caricamento delle proiezioni.

Le proiezioni potranno essere caricate solo dopo film e sale, perché ogni proiezione farà riferimento a un film e a una sala tramite identificativo.

Formato dati richiesto

I file saranno testuali e useranno il carattere ; come separatore principale dei campi.

Nel caso di valori multipli, come generi, tag o caratteristiche, verrà usato il carattere , .

Esempio per i film:

1;Dune Part Two;Denis Villeneuve;166;2024-03-01;FANTASCIENZA,AZIONE

Esempio per le sale:

1;Sala IMAX;220;true;IMAX,DOLBY_ATMOS,SCHERMO_GRANDE

Esempio per le proiezioni:

1;STANDARD;1;1;2026-06-10;21:30;10.00;-;-;SERALE,WEEKEND

Il sistema dovrà essere in grado di riconoscere diverse tipologie di proiezione a partire dal campo **tipo**.

Le tipologie previste sono:

STANDARD

TRE_D

EVENTO

Gestione degli errori di importazione

Se una riga del file contiene dati non validi, il sistema non deve interrompere completamente il caricamento.

Deve invece:

- segnalare il problema;
- ignorare la riga non valida;
- continuare a leggere le righe successive.

Esempi di errori da segnalare:

Riga 4 ignorata: genere non valido.

Riga 7 ignorata: sala non trovata.

Riga 10 ignorata: prezzo base non valido.

Il cliente non richiede, in questa fase, un sistema avanzato di log.
È sufficiente stampare i messaggi in console.

18. Report finali attesi

Alla fine del programma, il `main` deve stampare alcuni report per dimostrare che il gestionale funziona.

I report minimi richiesti sono:

1. catalogo completo dei film;
2. elenco completo delle sale;
3. programmazione completa;
4. proiezioni di oggi;
5. proiezioni future;
6. proiezioni di una data specifica;
7. proiezioni di una sala specifica;
8. film di un certo genere;
9. proiezioni serali;
10. proiezioni del weekend;
11. proiezioni con prezzo superiore a una certa soglia;
12. film ordinati per durata decrescente;
13. proiezioni ordinate per data e ora;
14. proiezioni ordinate per prezzo finale;
15. dettagli specifici delle proiezioni usando `instanceof`.

19. Validazioni richieste

Il sistema deve avere controlli semplici sui dati. Non serve bloccare il programma con errori complessi. Però non devono essere accettati dati chiaramente non validi.

Esempi:

- titolo film vuoto;
- regista vuoto;
- durata minore o uguale a 0;
- data di uscita mancante;
- nome sala vuoto;
- numero posti minore o uguale a 0;
- proiezione senza film;
- proiezione senza sala;
- prezzo base minore o uguale a 0;

- data proiezione mancante;
- ora proiezione mancante;
- genere mancante;
- tag vuoto.

In questi casi il programma deve stampare un messaggio chiaro.

Esempio:

Errore: il titolo del film non può essere vuoto.

oppure:

Errore: impossibile creare una proiezione senza sala.

20. Indicazioni non vincolanti sulla struttura

Il cliente non impone i nomi delle classi, ma una struttura sensata potrebbe prevedere:

Film
GenereFilm
Sala
Proiezione
ProiezioneStandard
Proiezione3D
EventoSpeciale
Archivio
GestoreCinema
Main

E alcune interfacce:

Identificabile
Programmabile
Prezzabile
Filtro
Notificatore

Questa è una possibile organizzazione coerente con le richieste.